

ANALISIS STUDI KASUS: DISCORD DAN WIDE-COLUMN STORE

1. Latar Belakang Perusahaan

Discord merupakan platform komunikasi *real-time* yang mengintegrasikan fitur pesan teks, panggilan suara, serta konferensi video dalam satu ekosistem terpadu. Didirikan pada tahun 2015, platform ini pada mulanya dirancang untuk memfasilitasi komunikasi dalam komunitas permainan daring (*gaming*). Namun seiring berjalannya waktu, Discord telah bertransformasi menjadi sarana komunikasi yang digunakan secara luas oleh berbagai kalangan, termasuk mahasiswa, pengembang perangkat lunak, komunitas kreatif, hingga entitas korporasi.

Hingga saat ini, Discord melayani lebih dari 200 juta pengguna aktif bulanan yang tersebar di 19 juta server aktif per minggu. Volume pesan yang tersimpan dalam sistem telah melampaui angka triliunan record. Skala operasional sebesar ini menimbulkan tantangan fundamental dalam hal penyimpanan dan pengambilan data secara efisien, khususnya pada subsistem penyimpanan riwayat pesan (*message history*) yang merupakan fitur inti dari platform.

2. Permasalahan yang Dihadapi Discord

2.1 Fase MongoDB (2015)

Pada tahap awal pengembangan, Discord mengadopsi MongoDB sebagai basis data utama. Pemilihan ini didasarkan pada kecepatan proses prototyping serta fleksibilitas format dokumen JSON yang ditawarkan oleh MongoDB. Pada skala awal, sistem berjalan dengan baik tanpa kendala berarti.

Namun pada November 2015, ketika akumulasi pesan mencapai 100 juta record, MongoDB mulai menunjukkan degradasi performa yang signifikan. Permasalahan utama yang teridentifikasi meliputi:

1. Data dan indeks tidak lagi dapat ditampung sepenuhnya dalam RAM, sehingga sistem harus mengakses disk untuk menjawab query yang seharusnya bersifat instan.
2. Latency query menjadi tidak stabil dan tidak dapat diprediksi secara konsisten.
3. Sistem mengalami kesulitan dalam melakukan skalabilitas secara linear seiring pertumbuhan data.

4. Beban operasi tulis (write) dan baca (read) meningkat secara drastis tanpa solusi yang memadai.

2.2 Karakteristik Data yang Mendasari Kebutuhan

Sistem pesan Discord memiliki karakteristik yang sangat spesifik dan menjadi pertimbangan utama dalam pemilihan arsitektur basis data:

1. **Write-heavy workload.** Setiap detik, ribuan pesan baru ditulis ke dalam basis data secara simultan dari jutaan channel yang berbeda. Rasio operasi tulis jauh melampaui operasi baca.
2. **Pertumbuhan data tanpa batas.** Pesan yang telah dikirimkan tidak dihapus secara otomatis oleh sistem. Data terakumulasi secara kontinu dari hari ke hari, bulan ke bulan, tanpa titik saturasi.
3. **Pola akses yang terprediksikan.** Pengguna hampir selalu mengakses data dengan pola yang seragam, yaitu membuka suatu channel dan membaca pesan terbaru. Query yang dijalankan bersifat konsisten: "ambil N pesan terakhir dari channel X."
4. **Tidak memerlukan operasi JOIN.** Setiap pesan bersifat mandiri dan mengandung seluruh informasi yang dibutuhkan (channel_id, author_id, content, timestamp) tanpa perlu digabungkan dengan tabel lain.
5. **Ketersediaan tinggi (High Availability).** Platform tidak boleh mengalami downtime. Apabila basis data mengalami kegagalan, ratusan juta pengguna tidak dapat mengakses layanan komunikasi.

3. Mengapa Discord Memilih Wide-Column Store

Pada tahun 2017, tim engineering Discord melakukan evaluasi menyeluruh terhadap berbagai alternatif basis data. Mereka menetapkan empat kriteria fundamental yang harus dipenuhi:

1. **Scalable** yaitu mampu ditingkatkan kapasitasnya secara horizontal seiring pertumbuhan data.
2. **Fault-tolerant** yaitu tetap beroperasi meskipun terjadi kegagalan pada sebagian node.

3. **Low maintenance** yaitu tidak memerlukan sumber daya manusia yang berlebihan untuk pengelolaan operasional.
4. **Bukan blob store** yaitu mampu melakukan query terhadap data secara langsung tanpa proses serialisasi yang mahal.

Apache Cassandra, yang merupakan implementasi dari paradigma Wide-Column Store, memenuhi seluruh kriteria tersebut. Berikut merupakan analisis keunggulan spesifik Cassandra yang relevan dengan kebutuhan Discord:

3.1 Arsitektur Peer-to-Peer

Cassandra mengadopsi arsitektur ring berbasis peer-to-peer, di mana seluruh node memiliki kedudukan yang setara. Tidak terdapat konsep master-slave yang lazim ditemui pada basis data relasional. Setiap node mampu menerima operasi baca maupun tulis, sehingga mengeliminasi single point of failure. Apabila satu node mengalami kegagalan, node lain yang menyimpan replika data akan mengambil alih pelayanan secara otomatis.

3.2 Linear Scalability

Penambahan kapasitas pada Cassandra dilakukan dengan menambahkan node baru ke dalam cluster. Performa sistem meningkat secara linear seiring penambahan node, tanpa memerlukan redesign arsitektur atau migrasi data manual. Pada kasus Discord, cluster berkembang dari 12 node pada tahap awal hingga mencapai 177 node seiring pertumbuhan volume data.

3.3 Write Performance Tinggi

Cassandra mengimplementasikan Log-Structured Merge Tree (LSM-tree) sebagai mekanisme penulisan data. Setiap operasi tulis hanya perlu melakukan *append* ke commit log dan memtable, tanpa perlu melakukan pencarian lokasi penyimpanan terlebih dahulu. Arsitektur ini menghasilkan throughput tulis yang sangat tinggi, mampu menangani ribuan operasi insert per detik.

3.4 Replikasi dan Tunable Consistency

Data pada Cassandra secara otomatis direplikasi ke beberapa node berdasarkan replication factor yang dikonfigurasi. Discord menggunakan replication factor sebesar 3, yang berarti setiap data memiliki tiga salinan di tiga node berbeda. Tingkat konsistensi (consistency level) dapat disesuaikan per operasi. Discord menggunakan QUORUM consistency, yang memberikan keseimbangan optimal antara konsistensi data dan performa sistem.

4. Model Data Pesan Discord di Cassandra

Aspek terpenting dari implementasi Cassandra di Discord terletak pada desain model data. Berbeda dengan basis data relasional yang menerapkan pendekatan entity-driven design, Cassandra mengharuskan pendekatan query-driven design, di mana struktur tabel dirancang berdasarkan pola query yang akan dijalankan.

4.1 Skema Awal (Versi Pertama)

Pada implementasi awal, Discord mendesain tabel pesan dengan skema sebagai berikut:

PRIMARY KEY (channel_id, message_id)

WITH CLUSTERING ORDER BY (message_id DESC)

Pada skema ini, **channel_id** berfungsi sebagai partition key yang menentukan di node mana data disimpan, sedangkan **message_id** berfungsi sebagai clustering key yang menentukan urutan data dalam partisi. Discord menggunakan Snowflake ID sebagai message_id karena bersifat unik dan sudah terurut secara kronologis.

Permasalahan yang muncul dari skema ini adalah partisi dapat bertumbuh tanpa batas. Channel yang sangat aktif (server dengan ratusan ribu anggota) akan terus menambahkan pesan ke partisi yang sama, menyebabkan ukuran partisi membesar secara tidak terkendali dan memperlambat operasi baca.

4.2 Skema Final (Versi Kedua) dengan Bucketing

Untuk mengatasi permasalahan Large Partition, Discord menambahkan konsep bucket, yaitu jendela waktu statis selama 10 hari. Setiap bucket membentuk partisi baru sehingga ukuran partisi tetap terkendali.

PRIMARY KEY ((channel_id, bucket), message_id)

WITH CLUSTERING ORDER BY (message_id DESC)

Partition key kini merupakan gabungan dari **channel_id** dan **bucket**. Seluruh pesan dari satu channel dalam rentang 10 hari tersimpan di partisi yang sama. Clustering key tetap menggunakan **message_id** dengan urutan descending, sehingga pesan terbaru berada di awal partisi dan dapat dibaca dengan sangat efisien.

Mekanisme perhitungan bucket didasarkan pada Discord Epoch (1 Januari 2015). Nilai bucket dihitung dengan membagi selisih waktu saat ini dengan Discord Epoch terhadap ukuran bucket (10 hari dalam milidetik). Dengan demikian, setiap 10 hari secara otomatis terbentuk partisi baru.

5. Arsitektur Sistem Discord

5.1 Arsitektur Awal (2017 hingga 2020)

Pada fase awal implementasi Cassandra, arsitektur Discord bersifat sederhana: API monolith yang ditulis dalam Python berkomunikasi langsung dengan cluster Cassandra. Tidak terdapat layer perantara di antara keduanya. Konfigurasi ini memadai untuk skala awal, namun menimbulkan permasalahan seiring pertumbuhan traffic.

5.2 Arsitektur Baru dengan Data Services (2020 dan seterusnya)

Discord menambahkan layer perantara bernama Data Services yang ditulis dalam bahasa pemrograman Rust. Layer ini memiliki dua fungsi kunci yang sangat penting:

Request Coalescing. Apabila terdapat 100 pengguna yang membuka channel yang sama secara bersamaan (misalnya ketika seseorang mengirim pesan @everyone di server besar), Data Services menggabungkan 100 request tersebut menjadi satu query tunggal ke basis data. Mekanisme ini secara signifikan mengurangi beban yang diterima oleh Cassandra.

Consistent Hash-based Routing. Seluruh request untuk channel yang sama diarahkan ke instance Data Services yang sama menggunakan channel_id sebagai routing key. Mekanisme ini meningkatkan efektivitas request coalescing karena request yang serupa terkonsentrasi pada satu instance.

Kedua mekanisme ini merupakan solusi arsitektural yang menangani stress management pada sistem. Alih-alih mengandalkan basis data untuk menangani seluruh lonjakan traffic secara langsung, Discord memperkenalkan layer intermediasi yang mengelola dan mengoptimalkan aliran request sebelum mencapai basis data.

6. Permasalahan Operasional pada Skala Triliun

Meskipun Cassandra beroperasi dengan baik selama beberapa tahun pertama, pada awal 2022 ketika cluster telah membengkak menjadi 177 node dengan triliunan pesan, muncul empat permasalahan operasional yang signifikan:

6.1 Hot Partition

Hot partition merupakan kondisi di mana satu partisi menerima traffic yang jauh melampaui kapasitas normalnya. Pada konteks Discord, hal ini terjadi ketika seseorang mengirim pesan @everyone di server besar dengan ratusan ribu anggota. Dalam hitungan detik, seluruh anggota membuka channel yang sama dan meminta data dari partisi yang sama. Node yang menyimpan partisi tersebut menjadi overload, dan karena Cassandra menggunakan quorum consistency, node lain yang menyimpan replika juga terdampak. Akibatnya, latency meningkat di seluruh cluster.

6.2 Tombstone Overhead

Pada Cassandra, data yang dihapus tidak langsung dihilangkan dari penyimpanan. Sebagai gantinya, Cassandra menulis penanda khusus bernama tombstone yang memberitahukan seluruh node bahwa data tersebut telah dihapus. Tombstone baru benar-benar dihapus setelah proses compaction dan masa berlakunya habis.

Discord menemukan permasalahan tambahan terkait tombstone. Skema mereka memiliki 16 kolom, namun rata-rata pesan hanya mengisi 4 kolom. Karena seluruh operasi tulis di Cassandra bersifat upsert, menulis nilai null pada kolom yang tidak terisi sama dengan membuat tombstone. Akibatnya, setiap operasi INSERT secara tidak disengaja menghasilkan 12 tombstone yang tidak diperlukan.

6.3 Compaction Lag

Cassandra secara berkala menggabungkan berkas-berkas di disk (SSTable) melalui proses yang disebut compaction. Proses ini esensial untuk menjaga performa operasi baca. Pada skala Discord, compaction sering tertinggal karena data yang harus diproses terakumulasi lebih cepat daripada kemampuan compaction. Konsekuensinya, operasi baca harus memeriksa semakin banyak SSTable, menghasilkan cascading latency di seluruh sistem.

6.4 JVM Garbage Collection

Cassandra diimplementasikan menggunakan bahasa pemrograman Java dan berjalan di atas Java Virtual Machine (JVM). JVM memiliki mekanisme garbage collector yang secara berkala membersihkan memori. Ketika garbage collector berjalan, seluruh operasi pada node yang bersangkutan terhenti sementara (stop-the-world pause). Pada skala Discord, jeda ini dapat berlangsung hingga beberapa detik, yang cukup signifikan untuk menyebabkan degradasi pengalaman pengguna.

7. Solusi dan Optimasi yang Diterapkan

7.1 Optimasi Tombstone

Discord menerapkan tiga langkah strategis untuk mengatasi permasalahan tombstone. Pertama, sistem dimodifikasi agar hanya menulis kolom yang benar-benar berisi data, mengeliminasi 12 tombstone yang tidak diperlukan per operasi INSERT. Kedua, lifespan tombstone dikurangi dari default 10 hari menjadi 2 hari, yang dimungkinkan karena Discord menjalankan Cassandra repair setiap malam. Ketiga, kode query dioptimasi untuk mencatat bucket yang telah kosong dan melewatinya pada query berikutnya.

7.2 Mitigasi Hot Partition

Permasalahan hot partition ditangani melalui implementasi Request Coalescing dan Consistent Hash-based Routing pada layer Data Services. Kedua mekanisme ini telah diuraikan pada Bab 5.2.

7.3 Migrasi ke ScyllaDB

Untuk mengatasi permasalahan fundamental yang berakar pada JVM garbage collection dan keterbatasan performa Cassandra pada skala triliun, Discord melakukan migrasi ke

ScyllaDB. ScyllaDB merupakan basis data wide-column yang sepenuhnya kompatibel dengan Cassandra (menggunakan CQL, model data, dan driver yang sama) namun diimplementasikan ulang menggunakan bahasa C++ dengan arsitektur shard-per-core.

8. Hasil Migrasi dan Perbandingan Performa

Proses migrasi dilaksanakan pada Mei 2022. Discord menulis ulang data migrator menggunakan bahasa Rust dan berhasil memigrasikan triliunan pesan dalam waktu 9 hari dengan kecepatan hingga 3,2 juta pesan per detik, tanpa downtime.

Hasil yang diperoleh setelah migrasi sangat signifikan:

Metrik	Cassandra	ScyllaDB
Jumlah node	177	72
Read latency (p99)	40 hingga 125 ms	15 ms
Write latency (p99)	5 hingga 70 ms	5 ms (stabil)
GC pause	Sering, hingga 10+ detik	Tidak ada (C++)
Beban operasional	Tinggi, sering firefighting	Rendah, stabil

Perlu ditekankan bahwa migrasi ke ScyllaDB tidak mengubah paradigma basis data yang digunakan. ScyllaDB tetap merupakan Wide-Column Store dengan model data dan query language yang identik dengan Cassandra. Perpindahan ini semata-mata merupakan penggantian engine ke implementasi yang lebih efisien pada lapisan infrastruktur.

9. Kesimpulan Studi Kasus

Studi kasus Discord mendemonstrasikan bahwa paradigma Wide-Column Store merupakan solusi yang sangat tepat untuk sistem penyimpanan data berskala masif dengan karakteristik write-heavy, time-series, dan pola akses yang terprediksikan. Cassandra berhasil membantu Discord bertransisi dari miliaran ke triliunan pesan, sementara ScyllaDB meningkatkan efisiensi dan performa ketika skala semakin membesar.

Pelajaran fundamental yang dapat ditarik dari studi kasus ini mencakup pentingnya query-driven data modeling, kebutuhan akan mekanisme bucketing untuk mencegah Large Partition, dampak negatif dari tombstone yang tidak terkelola, serta nilai strategis dari layer intermediasi (Data Services) dalam mengelola beban traffic sebelum mencapai basis data.

Referensi

- [1] D. Meas, "How Discord Creates Insights from Trillions of Data Points," discord.com, Nov. 08, 2021.
- [2] B. Ingram, "How Discord Stores Trillions of Messages," discord.com, Mar. 06, 2023.
- [3] V. Niu, "How Discord Indexes Trillions of Messages," discord.com, May 02, 2025.
- [4] R. O. Bray, "How Discord Processes 30+ Petabytes of Data," Betterstack.com, Sep. 25, 2024.
- [5] S. Chhabra, "Why Did Discord Go From MongoDB To Cassandra Then ScyllaDB?" shubhdeepchhabra.in, 2023.